JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicReferenceFieldUpdater<T,V>

java.lang.Object
  java.util.concurrent.atomic.AtomicReferenceFieldUpdater<T,V>

Type Parameters:

`T - The type of the object holding the updatable field`

`V - The type of the field`

```
public abstract class AtomicReferenceFieldUpdater<T,V>
extends Object
```

A reflection-based utility that enables atomic updates to designated `volatile` reference fields of designated classes. This class is designed for use in atomic data structures in which several reference fields of the same node are independently subject to atomic updates. For example, a tree node might be declared as

```
class Node {
  private volatile Node left, right;

  private static final AtomicReferenceFieldUpdater<Node, Node> leftUpdater =
    AtomicReferenceFieldUpdater.newUpdater(Node.class, Node.class, "left");
  private static AtomicReferenceFieldUpdater<Node, Node> rightUpdater =
    AtomicReferenceFieldUpdater.newUpdater(Node.class, Node.class, "right");

  Node getLeft() { return left; }
  boolean compareAndSetLeft(Node expect, Node update) {
    return leftUpdater.compareAndSet(this, expect, update);
  }
  // ... and so on
}
```

Note that the guarantees of the `compareAndSet` method in this class are weaker than in other atomic classes. Because this class cannot ensure that all uses of the field are appropriate for purposes of atomic access, it can guarantee atomicity only with respect to other invocations of `compareAndSet` and `set` on the same updater.

Since:

1.5

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AtomicReferenceFieldUpdater()`	Protected do-nothing constructor for use by subclasses.

Method Summary

Concrete

Modifier and Type	Method
`V`	`accumulateAndGet(T obj, V x, BinaryOperator<V> accumulatorFunct`

abstract boolean	**compareAndSet**(**T** obj, **V** expect, **V** update)
abstract **V**	**get**(**T** obj)
V	**getAndAccumulate**(**T** obj, **V** x, **BinaryOperator**<**V**> accumulatorFunct

V	**getAndSet**(**T** obj, **V** newValue)
V	**getAndUpdate**(**T** obj, **UnaryOperator**<**V**> updateFunction)
abstract void	**lazySet**(**T** obj, **V** newValue)
static <U,W> **AtomicReferenceFieldUpdater**<U,W>	

	newUpdater(**Class**<U> tclass, **Class**<W> vclass, **String** fieldName)
abstract void	**set**(**T** obj, **V** newValue)
V	**updateAndGet**(**T** obj, **UnaryOperator**<**V**> updateFunction)
abstract boolean	**weakCompareAndSet**(**T** obj, **V** expect, **V** update)

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AtomicReferenceFieldUpdater

```
protected AtomicReferenceFieldUpdater()
```

Protected do-nothing constructor for use by subclasses.

Method Detail

newUpdater

```
public static <U,W> AtomicReferenceFieldUpdater<U,W> newUpdater(Class<U> tclass,
                                                                Class<W> vclass,
                                                                String fieldName)
```

Creates and returns an updater for objects with the given field. The Class arguments are needed to check that reflective types and generic types match.

Type Parameters:

U - the type of instances of tclass

W - the type of instances of vclass

Parameters:

tclass - the class of the objects holding the field

vclass - the class of the field

fieldName - the name of the field to be updated

Returns:

the updater

Throws:

ClassCastException - if the field is of the wrong type

IllegalArgumentException - if the field is not volatile

RuntimeException - with a nested reflection-based exception if the class does not hold field or is the wrong type, or the field is inaccessible to the caller according to Java language access control

compareAndSet

```
public abstract boolean compareAndSet(T obj,
                                      V expect,
                                      V update)
```

Atomically sets the field of the given object managed by this updater to the given updated value if the current value == the expected value. This method is guaranteed to be atomic with respect to other calls to `compareAndSet` and `set`, but not necessarily with respect to other changes in the field.

Parameters:

obj - An object whose field to conditionally set

expect - the expected value

update - the new value

Returns:

true if successful

weakCompareAndSet

```
public abstract boolean weakCompareAndSet(T obj,
                                          V expect,
                                          V update)
```

Atomically sets the field of the given object managed by this updater to the given updated value if the current value == the expected value. This method is guaranteed to be atomic with respect to other calls to `compareAndSet` and `set`, but not necessarily with respect to other changes in the field.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

obj - An object whose field to conditionally set

expect - the expected value

update - the new value

Returns:

true if successful

set

```
public abstract void set(T obj,
                         V newValue)
```

Sets the field of the given object managed by this updater to the given updated value. This operation is guaranteed to act as a volatile store with respect to subsequent invocations of `compareAndSet`.

Parameters:

obj - An object whose field to set

newValue - the new value

lazySet

```
public abstract void lazySet(T obj,
                             V newValue)
```

Eventually sets the field of the given object managed by this updater to the given updated value.

Parameters:

obj - An object whose field to set

newValue - the new value

Since:

1.6

get

```
public abstract V get(T obj)
```

Gets the current value held in the field of the given object managed by this updater.

Parameters:

obj - An object whose field to get

Returns:

the current value

getAndSet

```
public V getAndSet(T obj,
                   V newValue)
```

Atomically sets the field of the given object managed by this updater to the given value and returns the old value.

Parameters:

`obj - An object whose field to get and set`

`newValue - the new value`

Returns:

`the previous value`

getAndUpdate

```
public final V getAndUpdate(T obj,
                            UnaryOperator<V> updateFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`obj - An object whose field to get and set`

`updateFunction - a side-effect-free function`

Returns:

`the previous value`

Since:

`1.8`

updateAndGet

```
public final V updateAndGet(T obj,
                            UnaryOperator<V> updateFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`obj - An object whose field to get and set`

updateFunction - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final V getAndAccumulate(T obj,
                                V x,
                                BinaryOperator<V> accumulatorFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

obj - An object whose field to get and set

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final V accumulateAndGet(T obj,
                                V x,
                                BinaryOperator<V> accumulatorFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

obj - An object whose field to get and set

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.